Python Calculator

A tkinter desktop app — project documentation

1. Overview

This project is a small desktop calculator written in Python with tkinter. It was built step by step as a way to review core Python skills: functions, error handling, modules, and building a graphical interface.

The code is deliberately split into two parts:

- `calculator_functions.py` holds the math. It has no interface code, so every function can be tested on its own.
- `main.py` builds the window and connects its buttons to those functions.

Features:

- The four basic operations: addition, subtraction, multiplication and division
- Power (exponent) operation
- Temperature conversion between Celsius, Fahrenheit and Kelvin using two dropdowns
- Input validation: empty fields, text instead of numbers, division by zero, invalid powers and oversized results show a message instead of crashing
- Clean result display: floating-point noise is rounded away and whole numbers appear without ".0"
- Custom window icon, color palette and layout

2. Project structure

All files live in one folder. `main.py` is the file you run.

File	Purpose
`main.py`	The graphical interface and the glue code that connects widgets to the functions.
`calculator_functions.py`	All calculation logic: arithmetic, power, and the six temperature conversions. Raises `ValueError` for invalid math.
`app_icon.png`	The window icon. It must be PNG (or GIF) because `tk.PhotoImage` cannot read JPG.

3. How it works

When the user clicks an operation button, this is what happens:

1. The user types two numbers into the entries (`entry1` and `entry2`) and clicks a button such as +.
2. The button's `command` is a `lambda` that calls `calculate(cf.add)`. The lambda delays the call until the click actually happens.
3. `calculate(operation)` reads both entries and converts their text to floats. If that fails, it shows a message and stops.
4. It calls the operation it was given. A `ValueError` (for example division by zero) or an `OverflowError` is turned into a message and the function stops.
5. `show_result` rounds the value, removes a trailing ".0" and writes it into the result label.

The temperature button follows the same pattern. `temperature_transition` reads the two dropdowns and `entry1`, looks up the right function in the `conversions` dictionary, and passes the answer to `show_result`.

4. calculator_functions.py

Every function takes numbers and returns a number. Anything that is mathematically invalid is reported with `raise ValueError(...)` and a readable message, so the interface only has to catch the error and display it. Celsius is the base unit for temperatures: Kelvin and Fahrenheit convert into each other by going through Celsius, so only four direct formulas are needed.

Function	What it does	Raises
`add(num1, num2)`	`num1 + num2`	—
`subtract(num1, num2)`	`num1 - num2`	—
`multiply(num1, num2)`	`num1 * num2`	—
`divide(num1, num2)`	`num1 / num2`	`ValueError` if `num2` is 0
`power(num1, num2)`	`num1 ** num2`	`ValueError` for 0 to a negative power, or a negative base with a non-integer power
`C_to_K(value)`	`value + 273.15`	—
`K_to_C(value)`	`value - 273.15`	—
`C_to_F(value)`	`value * 1.8 + 32`	—
`F_to_C(value)`	`(value - 32) / 1.8`	—
`K_to_F(value)`	Kelvin to Celsius to Fahrenheit	—
`F_to_K(value)`	Fahrenheit to Celsius to Kelvin	—

Full source

```
# ------------------------------------------------------------
# Basic arithmetic
# ------------------------------------------------------------

def add(num1, num2):
    return num1 + num2


def subtract(num1, num2):
    return num1 - num2


def multiply(num1, num2):
    return num1 * num2


def divide(num1, num2):
    # Division by zero is not allowed: raise an error with a readable message
    if num2 == 0:
        raise ValueError("تقسیم بر صفر مجاز نیست")
    return num1 / num2


def power(num1, num2):
    # 0 to a negative power means dividing by zero
    if num1 == 0 and num2 < 0:
        raise ValueError("صفر به توان منفی تعریف نشده است")

    # A negative base with a non-integer power gives a complex number
    if num1 < 0 and not float(num2).is_integer():
        raise ValueError("پایه‌ی منفی با توان غیرصحیح پشتیبانی نمی‌شود")

    return num1 ** num2


# ------------------------------------------------------------
# Temperature conversion (Celsius is the "base" unit)
# ------------------------------------------------------------

def C_to_K(value):
    return value + 273.15


def K_to_C(value):
    return value - 273.15


def C_to_F(value):
    return value * 1.8 + 32


def F_to_C(value):
    return (value - 32) / 1.8


def K_to_F(value):
```

```
    # Kelvin -> Celsius -> Fahrenheit
    c = K_to_C(value)
    return C_to_F(c)


def F_to_K(value):
    # Fahrenheit -> Celsius -> Kelvin
    c = F_to_C(value)
    return C_to_K(c)
```

5. main.py

The window is 400 by 600 pixels. Widgets are stacked top to bottom with `pack()`. Where two widgets need to sit side by side, they are placed inside a `Frame` and packed with `side='left'`.

Section	Widgets	Notes
Window	`Tk`, title, icon, `geometry`	The icon path is built from `__file__`, so the program works no matter which folder it is launched from.
Welcome label	`Label`	`fill='x'` stretches it across the full width.
Number entries	`Frame` + two `Entry`	The frame is the parent of both entries; `side='left'` puts them on one line.
Result label	`Label`	Starts empty; its text is changed with `config(text=...)`.
Operation buttons	Two `Frame` + five `Button`	Each `command` is a `lambda` that calls `calculate` with a function from `cf`.
Temperature	`Frame` + two `ttk.Combobox` + `Button`	`state='readonly'` means the user can only pick from the list.

Color palette

Swatch	Hex	Used for
	`#0077B6`	Welcome label background
	`#0096c7`	Entry background
	`#00B4D8`	Result label background
	`#48CAE4`	Button background
	`#CAF0F8`	Text cursor color in the entries

Full source

```
# Import the tkinter library and give it the short name "tk"
import tkinter as tk

# ttk provides the modern widgets (we need its Combobox / dropdown)
from tkinter import ttk

# The os module helps us build file paths
import os

# Our own math functions, imported with the short name "cf"
import calculator_functions as cf

# ------------------------------------------------------------
# Main window
# ------------------------------------------------------------

# Create the main window of the application
window = tk.Tk()

# Set the title shown at the top of the window
window.title("ماشین حساب")

# Build the full path to the icon, which sits in the same folder as main.py
base_dir = os.path.dirname(os.path.abspath(__file__))
icon_path = os.path.join(base_dir, "app_icon.png")

# Load the icon image (PNG or GIF only) and use it as the window icon
icon = tk.PhotoImage(file=icon_path)
window.iconphoto(True, icon)

# Set the window size (width x height, in pixels)
window.geometry("400x600")

# ------------------------------------------------------------
# Welcome label
# ------------------------------------------------------------

label = tk.Label(window,
                 text='به برنامه ماشین حساب خوش آمدید',
                 fg='white',
                 font=('vazir', 14),  # (font name, size)
                 bg='#0077B6',
                 height=3)

# Stretch the label horizontally to fill the full width
label.pack(fill='x', pady=10, padx=5)

# ------------------------------------------------------------
# Number entries (side by side inside one frame)
# ------------------------------------------------------------

# A frame is an invisible box that holds both entries on one line
entries_frame = tk.Frame(window)
entries_frame.pack(pady=10)

entry1 = tk.Entry(entries_frame,
```

```
                  width=15,                    # width in characters
                  fg='white',
                  font=('vazir', 14),
                  bg='#0096c7',
                  insertbackground='#CAF0F8')  # color of the text cursor
entry1.pack(side='left', padx=5)

entry2 = tk.Entry(entries_frame,
                  width=15,
                  fg='white',
                  font=('vazir', 14),
                  bg='#0096c7',
                  insertbackground='#CAF0F8')
entry2.pack(side='left', padx=5)

# ---------------------------------------------------------------
# Result label
# ---------------------------------------------------------------

result_label = tk.Label(window,
                        text='',
                        fg='white',
                        font=('vazir', 14),
                        bg='#00B4D8')
result_label.pack(pady=10, padx=27, fill='x')


# ---------------------------------------------------------------
# Logic that connects the interface to calculator_functions.py
# ---------------------------------------------------------------

# Format a result and show it in the result label
def show_result(result):
    # Round to 2 decimals to hide tiny floating-point errors (0.1 + 0.2 -> 0.3)
    result = round(result, 2)

    # If the result is a whole number (like 5.0), convert it to int so it shows as 5
    if result.is_integer():
        result = int(result)

    # Display the final value in the result label
    result_label.config(text=result)


# Read both entries, run the given operation on them, and show the result
def calculate(operation):
    # First try: convert the entries' text into numbers
    try:
        num1 = float(entry1.get())
        num2 = float(entry2.get())
    except ValueError:
        # The user left an entry empty or typed something that is not a number
        result_label.config(text="لطفاً دو عدد معتبر وارد کنید")
        return  # stop the function here

    # Second try: run the operation that was passed in
    try:
        result = operation(num1, num2)
```

```
    except ValueError as e:
        # e holds the message we wrote in calculator_functions (e.g. division by zero)
        result_label.config(text=str(e))
        return
    except OverflowError:
        # The result is too big for a float to hold
        result_label.config(text="نتیجه خیلی بزرگ است")
        return

    show_result(result)


# Map each (from unit, to unit) pair to the function that does the conversion
# The functions are written WITHOUT parentheses: we only store them here
conversions = {
    ("کلوین" ,"سلسیوس"): cf.C_to_K,
    ("سلسیوس" ,"کلوین"): cf.K_to_C,
    ("فارنهایت" ,"سلسیوس"): cf.C_to_F,
    ("سلسیوس" ,"فارنهایت"): cf.F_to_C,
    ("فارنهایت" ,"کلوین"): cf.K_to_F,
    ("کلوین" ,"فارنهایت"): cf.F_to_K,
}


# Read the dropdowns and the first entry, convert the temperature, and show it
def temperature_transition():
    # Read both combo boxes
    base, dest = combo1.get(), combo2.get()

    # Convert the first entry's text into a number
    try:
        value = float(entry1.get())
    except ValueError:
        result_label.config(text="لطفاً عدد معتبر وارد کنید")
        return

    # Same unit on both sides: nothing to convert
    if base == dest:
        result = value
    else:
        # Find the right function for this pair, then call it with the value
        result = conversions[(base, dest)](value)

    show_result(result)


# ---------------------------------------------------------------
# Operation buttons
# command gets a lambda so calculate() runs only when the button is clicked
# ---------------------------------------------------------------

operation1_frame = tk.Frame(window)
operation1_frame.pack(pady=10)

add_but = tk.Button(operation1_frame,
                    width=10,
                    text='+',
                    command=lambda: calculate(cf.add),
```

```
                    bg='#48CAE4',
                    fg='white',
                    font=('vazir', 14))
add_but.pack(side='left', padx=8)

sub_but = tk.Button(operation1_frame,
                    width=10,
                    text='-',
                    command=lambda: calculate(cf.subtract),
                    bg='#48CAE4',
                    fg='white',
                    font=('vazir', 14))
sub_but.pack(side='left', padx=8)

operation2_frame = tk.Frame(window)
operation2_frame.pack(pady=10)

mul_but = tk.Button(operation2_frame,
                    width=10,
                    text='x',
                    command=lambda: calculate(cf.multiply),
                    bg='#48CAE4',
                    fg='white',
                    font=('vazir', 14))
mul_but.pack(side='left', padx=8)

dvd_but = tk.Button(operation2_frame,
                    width=10,
                    text='÷',
                    command=lambda: calculate(cf.divide),
                    bg='#48CAE4',
                    fg='white',
                    font=('vazir', 14))
dvd_but.pack(side='left', padx=8)

pow_but = tk.Button(window,
                    width=10,
                    text='**',
                    command=lambda: calculate(cf.power),
                    bg='#48CAE4',
                    fg='white',
                    font=('vazir', 14))
pow_but.pack(pady=10)

# ------------------------------------------------------------------
# Temperature conversion: two dropdowns and a button
# ------------------------------------------------------------------

combobox_frame = tk.Frame(window)
combobox_frame.pack(pady=10)

# Dropdown for the unit we convert FROM (readonly = the user can only pick)
combo1 = ttk.Combobox(combobox_frame,
                      font=('vazir', 14),
                      width=10,
                      values=["فارنهایت" ,"کلوین" ,"سلسیوس"],
                      state="readonly")
combo1.current(0)  # select the first option by default
```

```
combo1.pack(side='left', padx=10)

# Dropdown for the unit we convert TO
combo2 = ttk.Combobox(combobox_frame,
                      font=('vazir', 14),
                      width=10,
                      values=["فارنهایت" ,"کلوین" ,"سلسیوس"],
                      state="readonly")
combo2.current(1)  # select a different option by default
combo2.pack(side='left', padx=10)

# The convert button; command gets the function name WITHOUT parentheses
temp_but = tk.Button(window,
                     width=10,
                     text='تبدیل دما',
                     command=temperature_transition,
                     bg='#48CAE4',
                     fg='white',
                     font=('vazir', 14))
temp_but.pack(pady=10)

# Keep the window open until the user closes it
window.mainloop()
```

6. Design decisions

- **Logic and interface are separate.** The functions know nothing about tkinter, so changing the look never risks breaking the math.
- **Validation lives with the logic.** `divide` and `power` raise errors themselves; the interface only catches and shows them.
- **One function for five operations.** `calculate(operation)` takes the operation as a parameter, so reading the entries and handling errors is written once, not five times.
- **One function for displaying results.** `show_result` is the only place that rounds and formats, so the display rule can be changed in one spot.
- **Rounding is for display only.** The functions keep full precision; rounding happens at the last step. Results are rounded to 2 decimals, so a very small answer such as 0.001 + 0.0004 shows as 0. Increase the second argument of `round()` if more precision is needed.
- **A dictionary replaces six `if` statements.** `conversions` maps each (from, to) pair of units to its function. The unit names in its keys must match the dropdown values exactly.

7. Python concepts practiced

Concept	Where it appears
Functions with parameters and `return`	Every function in `calculator_functions.py`
`raise` and custom error messages	`divide`, `power`

Concept	Where it appears
`try` / `except`, several `except` blocks	`calculate`, `temperature_transition`
`import` and `import ... as`	`import calculator_functions as cf`, `from tkinter import ttk`
Functions as values	`calculate(operation)`, the `conversions` dictionary
`lambda`	The operation buttons' `command`
Dictionaries with tuple keys	`conversions`
Type conversion	`float(entry1.get())`
Floating-point precision and rounding	`show_result`
tkinter widgets and layout	`Label`, `Entry`, `Button`, `Frame`, `ttk.Combobox`, `pack`

8. Error messages

These messages are shown in the result label instead of crashing the program. The interface text is in Persian.

Situation	Message shown	Meaning
Empty entry or text instead of a number (operations)	لطفاً دو عدد معتبر وارد کنید	Please enter two valid numbers
Empty entry or text instead of a number (temperature)	لطفاً عدد معتبر وارد کنید	Please enter a valid number
Division by zero	تقسیم بر صفر مجاز نیست	Division by zero is not allowed
Zero to a negative power	صفر به توان منفی تعریف نشده است	Zero to a negative power is undefined
Negative base with a non-integer power	پایه‌ی منفی با توان غیرصحیح پشتیبانی نمی‌شود	A negative base with a non-integer power is not supported
Result too large for a float	نتیجه خیلی بزرگ است	The result is too large

9. Running and testing

Keep `main.py`, `calculator_functions.py` and `app_icon.png` in the same folder. Run `main.py` with the Run button in VS Code, or from a terminal opened in that folder:

```
python main.py
```

Python 3 with tkinter is required; tkinter is included in the standard Windows installer. Try these cases to check everything works:

Input	Expected result
2 + 3	5
0.1 + 0.2	0.3
10 ÷ 0	Division by zero message
Empty entries, any operation	"Enter two valid numbers" message
-8 power 0.5	Negative base message
0 power -1	Zero to a negative power message
10 power 1000	"Result too large" message
-8 power 3	-512
100 Celsius to Fahrenheit	212
300 Kelvin to Celsius	26.85
Same unit in both dropdowns	The number is returned unchanged

10. Ideas for extending the project

- Press Enter to calculate (look up the `bind` method).
- Add a Clear button that empties the entries and the result.
- Reject temperatures below absolute zero (for example -300 Celsius).
- Replace the entries with a numeric keypad laid out with `grid`.
- Color the dropdowns to match the theme using `ttk.Style`.
- Package the app as an .exe with a tool such as PyInstaller.